MIFX — Machining Intent Format eXchange

Core Specification

MIFX Open Specification

Version 1.1 — March 12, 2026

Authorship

The MIFX specification was originally authored by Azlan Hadi.

MIFX is released as an open specification intended to enable neutral exchange of machining intent between manufacturing systems.

Contents

1 Introduction **6**

2 MIFX Package **7**
2.1 Entry Point . . . 7
2.2 Filesystem Organization . . . 7
2.3 Portability . . . 8

3 Object Model **9**
3.1 Primary Objects . . . 9
3.2 Object Identity . . . 9
3.3 Object References . . . 9
3.4 Object Independence . . . 10

4 Artifact System **11**
4.1 Artifact Structure . . . 11
4.2 Artifact Roles . . . 11
4.3 Artifact Kinds . . . 12
4.4 Relative Paths . . . 12
4.5 Optional Artifacts . . . 12

5 Job Object **13**
5.1 Job Structure . . . 13
5.2 MIFX Version . . . 14
5.3 Header . . . 14
5.4 Setup References . . . 14
5.5 Tool Groups . . . 15
5.6 Optional Addons . . . 15

6 Setup Object **16**
6.1 Setup Structure . . . 16
6.2 Setup Identifier . . . 17
6.3 Setup Index . . . 17
6.4 Setup Artifacts . . . 17
6.5 Operations . . . 18

7 Operation Object **19**
7.1 Operation Structure . . . 19
7.2 Operation Identifier . . . 20
7.3 Strategy . . . 20
7.4 Operation Artifacts . . . 20

8 Tool Object **21**
8.1 Tool Structure . . . 21
8.2 Tool Identifier . . . 22

8.3 Tool Artifacts . . . 22
8.4 Tool Profile . . . 22
8.5 Tool Mount . . . 22
8.6 Tool Controls . . . 23
8.7 Tool Extensions . . . 23

9 In-Process Object **24**
9.1 In-Process Structure . . . 24
9.2 Operation Coverage . . . 24
9.3 In-Process Artifacts . . . 25
9.4 Optional Nature . . . 25

10 Geometry Resources **26**
10.1 Geometry Referencing . . . 26
10.2 Supported Geometry Formats . . . 26
10.3 Setup Assemblies . . . 27
10.4 Part Geometry . . . 27
10.5 Stock Geometry . . . 27
10.6 Fixture Geometry . . . 27
10.7 Tool Geometry . . . 27
10.8 In-Process Geometry . . . 27

11 Extensions and Addons **28**
11.1 Extensions . . . 28
11.2 Addons . . . 28
11.3 Addon Referencing . . . 29
11.4 Extension Safety . . . 29

12 MIFX Package Format **30**
12.1 Root Structure . . . 30
12.2 Directory Overview . . . 30
12.3 Directory Flexibility . . . 31
12.4 Transport and Distribution . . . 31

13 Compatibility and Forward Evolution **32**
13.1 Unknown Fields . . . 32
13.2 Unknown Objects . . . 32
13.3 Extensions and Addons . . . 32
13.4 Version Identification . . . 32
13.5 Backward Compatibility . . . 32

A Minimal MIFX Package **33**
A.1 Directory Structure . . . 33
A.2 job.json . . . 33
A.3 setup.json . . . 34
A.4 op-1.json . . . 34

B Milling Operation Example **35**

C Tool Definition Example **36**

D Complete MIFX Package Structure **37**

1 Introduction

The **Machining Intent Format eXchange (MIFX)** is an open and neutral data format designed to exchange machining process information between manufacturing software systems.

Modern machining environments involve multiple systems including:

- CAM software
- verification and simulation tools
- shop floor visualization systems
- inspection and quality tools
- manufacturing lifecycle platforms

These systems typically rely on proprietary data models that are difficult to exchange or interpret outside their native environments.

MIFX provides a lightweight, structured format that captures the *intent of a machining process* in a portable and extensible package.

A MIFX package may contain:

- machining setups
- operations and toolpaths
- tool definitions
- setup and part geometry
- optional in-process states
- optional extensions

The format is designed with the following principles:

- **Neutral exchange** between systems
- **Human-readable data structures**
- **Extensible architecture**
- **Minimal core specification**
- **Long-term stability**

MIFX packages machining process data into a self-contained archive using a structured filesystem layout and JSON-based object descriptions.

This architecture allows tools to interpret only the parts of the package they understand while safely ignoring unknown extensions.

2 MIFX Package

A MIFX file is a self-contained archive containing machining process data and associated resources.

The archive uses the file extension:

```
.mifx
```

A MIFX package is a standard ZIP archive containing a structured filesystem layout.

2.1 Entry Point

The root of every MIFX package must contain a file named:

```
job.json
```

The `job.json` file acts as the entry point of the package and defines the primary structure of the machining job.

Software reading a MIFX package must begin interpretation from this file.

2.2 Filesystem Organization

The contents of a MIFX package are organized into directories representing different aspects of the machining process.

A typical package structure is shown below:

```
job.mifx
|
|-- job.json
|
|-- setups/
|   `-- set-1/
|       |-- setup.json
|       `-- operations/
|           |-- op-1.json
|           `-- op-2.json
|
|-- tools/
|-- geometry/
|-- in_process/
`-- addons/
```

Each directory contains objects and resources associated with the machining job.

The structure is designed to remain human-readable while allowing software systems to efficiently navigate the package.

2.3 Portability

A MIFX package is intended to be portable between systems without modification.

All files required to interpret the machining job should be included within the archive.

External references are discouraged to ensure long-term integrity and reproducibility of the data.

3 Object Model

MIFX represents machining data as a collection of independent objects stored as JSON files within the package.

Each object describes a specific aspect of the machining process and may reference additional data through artifacts.

This design allows systems to interpret only the parts of the package they understand while safely ignoring unsupported objects or extensions.

3.1 Primary Objects

The core MIFX object types are:

- **Job** — the root object describing the machining package
- **Setup** — a manufacturing setup configuration
- **Operation** — an individual machining operation
- **Tool** — a cutting tool definition
- **In-Process** — an optional intermediate workpiece state

Each object is stored as a JSON file and may reference additional resources through artifacts.

3.2 Object Identity

Each object must define an identifier using the `id` field.

Example:

```
{
  "id": "op-2"
}
```

Object identity is defined by the `id` field rather than by the filename. File names within the package are therefore not required to follow a specific naming convention.

3.3 Object References

Objects may reference other objects or resources using relative file paths.

Example:

```
{
  "id": "op-2",
  "artifacts": [
    {
      "role": "tool",
      "kind": "json",
      "path": "../../tools/main/2.json",
      "present": true
```

```
    }
  ]
}
```

This approach allows the package to behave as a lightweight object graph where nodes reference related resources.

3.4 Object Independence

Objects within a MIFX package are designed to be loosely coupled.

Software implementations may interpret only the objects and artifacts relevant to their functionality while ignoring unsupported elements.

This design enables interoperability between systems with different capabilities while maintaining forward compatibility.

4 Artifact System

Artifacts provide a mechanism for objects to reference additional data within a MIFX package.

An artifact describes the relationship between an object and an associated resource such as a toolpath file, geometry file, or auxiliary JSON payload.

Artifacts allow objects to remain lightweight while linking to external data stored elsewhere in the package.

4.1 Artifact Structure

Artifacts are defined as objects within an `artifacts` array.

Example:

```
{
  "role": "toolpath",
  "kind": "apt",
  "path": "cldata/op-2.apt",
  "present": true
}
```

Each artifact contains the following fields:

- **role** — describes the semantic purpose of the artifact
- **kind** — describes the format or data type of the artifact
- **path** — relative file path to the referenced resource
- **present** — indicates whether the referenced resource is included in the package

4.2 Artifact Roles

The `role` field defines how the referenced resource should be interpreted.

Examples of artifact roles include:

- `toolpath`
- `tool`
- `operation_csys`
- `setup_geometry`
- `tool_profile`
- `in_process_mesh`

The specification does not restrict artifact roles to a fixed list.

Software implementations should ignore roles they do not recognize.

4.3 Artifact Kinds

The `kind` field describes the data format of the referenced resource.

Examples include:

- `json`
- `apt`
- `stl`
- `3mf`
- `step`

Implementations may support additional formats depending on their capabilities.

4.4 Relative Paths

Artifact paths are specified using relative paths within the package.

This ensures that the entire MIFX archive remains self-contained and portable.

Example:

```
"path": "../../tools/main/2.json"
```

4.5 Optional Artifacts

The `present` field indicates whether the referenced resource exists in the package.

This allows a MIFX file to declare resources that may be generated or supplied later by other systems.

Software implementations should handle missing artifacts gracefully.

5 Job Object

The Job object represents the root of a MIFX package.

It provides the entry point for interpreting the machining process and defines the primary structure of the package.

The Job object is stored in the file:

`job.json`

located at the root of the MIFX archive.

5.1 Job Structure

A Job object typically contains:

- package metadata
- setup references
- tool group references
- optional addon references

Example:

```
{
  "mifxVersion": "0.1",

  "header": {
    "export": {
      "id": "d347ed61-2e99-4b69-8946-fcba14292330",
      "at": "2026-03-11T19:03:24Z",
      "by": "Fusion360",
      "exporter": "MIFXExporter"
    },
    "job": {
      "name": "3+2-Milling v5"
    }
  },

  "setups": [
    {
      "id": "set-1",
      "path": "setups/set-1/setup.json"
    }
  ],

  "tools": [
```

```
    {
      "role": "tool_group",
      "kind": "json",
      "path": "tools/main.json",
      "present": true
    }
  ]
}
```

5.2 MIFX Version

The `mifxVersion` field specifies the version of the MIFX specification used to create the package.

Example:

```
"mifxVersion": "0.1"
```

Software implementations should verify compatibility before processing the package.

5.3 Header

The `header` object provides metadata describing the origin of the package.

Typical information may include:

- exporting system
- export timestamp
- source design document
- job name

The structure of the header is intentionally flexible and may vary between implementations.

5.4 Setup References

The `setups` array lists the setups contained in the package.

Each entry contains:

- the setup identifier
- the relative path to the setup object

Example:

```
{
  "id": "set-1",
  "path": "setups/set-1/setup.json"
}
```

The order of setups in this array represents the logical order of setups in the machining process.

5.5 Tool Groups

Tool definitions are organized into tool groups located in the `tools` directory.

The Job object references tool groups through artifacts.

Example:

```
{
  "role": "tool_group",
  "kind": "json",
  "path": "tools/main.json",
  "present": true
}
```

Tool groups allow multiple machine configurations (such as milling spindles or lathe turrets) to be represented within the same job.

5.6 Optional Addons

The Job object may reference optional addon definitions that extend the package with additional functionality.

Addons are typically stored in the `addons` directory.

Implementations should ignore addon definitions they do not recognize.

6 Setup Object

A Setup object represents a machining setup within a MIFX package.

A setup describes the configuration in which machining operations are executed. This may include setup-level resources such as part, stock, fixture, setup offsets, notes, and setup assemblies.

Each setup is stored as an independent JSON object within the `setups` directory.

Example location:

```
setups/set-1/setup.json
```

6.1 Setup Structure

A Setup object typically contains:

- a unique identifier
- a setup index
- a descriptive name
- optional artifacts describing setup resources
- an ordered list of operations executed within the setup

Example:

```
{
  "id": "set-1",
  "index": 1,
  "name": "1st setup",

  "artifacts": [
    {
      "role": "notes",
      "kind": "json",
      "path": "notes.json",
      "present": true
    },
    {
      "role": "setup_offset",
      "kind": "json",
      "path": "offset.json",
      "present": true
    }
  ],

  "operations": [
    {
```

```
      "id": "op-1",
      "role": "toolpath",
      "kind": "mill",
      "path": "operations/op-1.json"
    },
    {
      "id": "op-2",
      "role": "toolpath",
      "kind": "mill",
      "path": "operations/op-2.json"
    }
  ]
}
```

6.2 Setup Identifier

Each setup must define an identifier using the `id` field.

Example:

```
"id": "set-1"
```

6.3 Setup Index

The `index` field indicates the logical order of the setup within the machining job.

Example:

```
"index": 1
```

This value is typically determined by the exporting CAM system.

6.4 Setup Artifacts

The Setup object may reference additional setup-level resources using artifacts.

Common setup artifacts include:

- notes
- setup offsets
- part definitions
- stock definitions
- fixture definitions
- setup assemblies

These resources are stored as separate files within the setup directory.

6.5 Operations

The `operations` array defines the machining operations executed in the setup.

Each entry contains:

- the operation identifier
- the operation role
- the operation kind
- the relative path to the Operation object

Example:

```
{
  "id": "op-2",
  "role": "toolpath",
  "kind": "mill",
  "path": "operations/op-2.json"
}
```

The order of entries in the `operations` array represents the execution order of the machining process.

7 Operation Object

An Operation object represents a machining operation within a setup.

Operations define the machining action performed on the workpiece and reference the resources required to execute that action, such as tools, toolpaths, coordinate systems, offsets, compensation, notes, and process parameters.

Each operation is stored as an independent JSON object within the `operations` directory of a setup.

Example location:

```
setups/set-1/operations/op-2.json
```

7.1 Operation Structure

An Operation object typically contains:

- an operation identifier
- an optional descriptive name
- a strategy identifier
- artifact references to associated resources

The operation role and machining kind are defined by the setup operation index entry rather than repeated within the operation object.

Example:

```
{
  "id": "op-2",
  "name": "Face1",
  "strategy": "face",

  "artifacts": [
    {
      "role": "tool",
      "kind": "json",
      "path": "../../../tools/main/2.json",
      "present": true
    },
    {
      "role": "toolpath",
      "kind": "apt",
      "path": "cldata/op-2.apt",
      "present": true
    },
    {
      "role": "operation_csys",
```

```
      "kind": "json",
      "path": "csys/op-2.json",
      "present": true
    }
  ]
}
```

7.2 Operation Identifier

Each operation must define a unique identifier using the `id` field.

Example:

```
"id": "op-2"
```

The identifier allows the operation to be indexed by the setup and referenced by other objects within the package.

7.3 Strategy

The `strategy` field identifies the machining strategy used by the operation.

Examples include:

- `face`
- `contour`
- `adaptive`
- `drill`

The strategy field is informational and may vary depending on the exporting CAM system.

7.4 Operation Artifacts

Operations reference associated resources through artifacts.

Common artifacts associated with operations include:

- tool definitions
- toolpath files
- coordinate system definitions
- cutting condition definitions
- compensation definitions
- operation offsets
- notes or documentation
- extensions and addons

These resources are stored as separate files within the operation directory.

Software implementations may interpret only the artifacts they support while ignoring unsupported elements.

8 Tool Object

A Tool object describes a cutting tool used by machining operations within a MIFX package.

Tools are defined independently from operations and may be referenced by multiple operations throughout the machining process.

Tool objects are stored within the `tools` directory and are organized into tool groups.

Example location:

```
tools/main/7.json
```

8.1 Tool Structure

A Tool object typically contains:

- a tool identifier
- optional descriptive metadata
- artifact references describing tool properties

Example:

```
{
  "id": "7",
  "unit": "in",
  "description": "1/2 Bull Mill",

  "artifacts": [
    {
      "role": "tool_profile",
      "kind": "json",
      "path": "profiles/7.json",
      "present": true
    },
    {
      "role": "tool_mount",
      "kind": "json",
      "path": "mounts/7.json",
      "present": true
    },
    {
      "role": "tool_controls",
      "kind": "json",
      "path": "controls/7.json",
      "present": true
    },
    {
      "role": "tool_extensions",
```

```
      "kind": "json",
      "path": "extensions/7.json",
      "present": true
    }
  ]
}
```

8.2 Tool Identifier

Each tool must define an identifier using the `id` field.

Example:

```
"id": "7"
```

Tool numbering and group context are defined by the tool group structure rather than by the filename.

8.3 Tool Artifacts

Tool properties are defined through artifacts that reference additional resources.

Common tool artifacts include:

- tool profile definitions
- tool mounting configuration
- control point definitions
- optional geometry meshes
- optional extensions

These resources are stored as separate files within the tool group directory.

8.4 Tool Profile

Tool profiles describe the geometric shape of the cutting tool.

Profiles may be defined using parametric descriptions or geometry files.

Examples include:

- revolved profiles
- line and arc segment definitions
- mesh representations

8.5 Tool Mount

Tools may optionally define mounting configurations describing how the tool is oriented relative to the machine spindle.

Mount definitions may include transformation matrices representing angle heads or other mounting devices.

8.6 Tool Controls

Tool controls describe control system references associated with the tool.

Examples include:

- length offsets
- diameter offsets
- control points used by the machining process

8.7 Tool Extensions

Additional tool information may be provided using extension artifacts.

Examples may include:

- holder descriptions
- vendor information
- tooling system identifiers

Implementations should ignore extensions they do not recognize.

9 In-Process Object

An In-Process object represents the intermediate state of the workpiece during a machining process.

In-process objects are optional and may be generated by simulation, verification, analysis tools, or package-building systems.

They provide a representation of the workpiece state after a specified sequence of operations.

In-process objects are stored within the `in_process` directory of the MIFX package.

Example location:

```
in_process/op1-5.json
```

9.1 In-Process Structure

An In-Process object typically contains:

- an identifier
- an optional descriptive name
- optional coverage information
- artifact references describing the workpiece state

Example:

```
{
  "id": "op1-5",
  "name": "After operations 1 to 5",

  "artifacts": [
    {
      "role": "in_process_mesh",
      "kind": "stl",
      "path": "meshes/op1-5.stl",
      "present": true
    }
  ]
}
```

9.2 Operation Coverage

In-process objects typically correspond to a range of operations that produced the represented workpiece state.

The naming of the object may indicate the covered operation range.

The specification does not require a fixed naming convention.

9.3 In-Process Artifacts

In-process objects reference associated resources through artifacts.

Common artifacts include:

- workpiece geometry meshes
- analysis metadata
- optional extensions
- optional addons

9.4 Optional Nature

In-process objects are optional within a MIFX package.

A valid MIFX file may omit the `in_process` directory entirely.

Systems that do not support in-process data may safely ignore these objects.

10 Geometry Resources

Geometry resources represent the physical objects involved in the machining process.

These may include:

- part models
- stock material
- fixtures
- setup assemblies

Geometry resources are stored within the `geometry` directory of the MIFX package.

Example structure:

```
geometry/
  setups/
  parts/
  stock/
  fixtures/
```

The specification does not require all geometry directories to be present.

10.1 Geometry Referencing

Geometry resources are referenced through artifact objects contained within MIFX objects such as setup resources.

Example:

```
{
  "role": "geometry",
  "kind": "3mf",
  "path": "../../geometry/setups/set-1.3mf",
  "present": true
}
```

The `path` field references the location of the geometry file within the MIFX package.

10.2 Supported Geometry Formats

The MIFX specification does not mandate a single geometry format.

Common formats include:

- STL
- 3MF
- STEP
- OBJ

Implementations may support additional formats depending on their capabilities.

10.3 Setup Assemblies

Setup assemblies represent the full machining configuration for a setup.

Assemblies are typically stored in the `geometry/setups` directory.

10.4 Part Geometry

Part models represent the design geometry of the component being manufactured.

Part geometry is typically stored in the `geometry/parts` directory.

10.5 Stock Geometry

Stock geometry describes the initial material from which the part will be machined.

Stock models are typically stored in the `geometry/stock` directory.

10.6 Fixture Geometry

Fixture models represent workholding or machine fixtures used during machining.

Fixture geometry is typically stored in the `geometry/fixtures` directory.

10.7 Tool Geometry

Tool geometry is not stored in the global `geometry` directory.

Optional tool meshes are stored within the corresponding tool group directory as part of the Tool object.

10.8 In-Process Geometry

Intermediate workpiece geometry is stored within the `in_process` directory alongside the corresponding In-Process object.

11 Extensions and Addons

MIFX allows additional information to be attached to objects through extensions and addons.

This mechanism enables organizations and software vendors to extend the format without modifying the core specification.

Systems that do not recognize an extension or addon must ignore it and continue processing the MIFX package.

11.1 Extensions

Extensions provide additional structured metadata stored as separate JSON resources associated with an object.

Extensions are typically stored in an `extensions` directory adjacent to the object domain.

Example structure:

```
setups/
  set-1/
    operations/
      extensions/
        op-1.json
        op-2.json
```

The meaning of extension fields is defined by the producing system.

Implementations that do not understand an extension must ignore it.

11.2 Addons

Addons provide larger or more structured external attachments stored as separate JSON resources.

Addons are placed inside an `addons` directory associated with a specific object or package domain.

Example structure:

```
setups/
  set-1/
    addons/
      setup-addon-1.json

    operations/
      addons/
        op-1.json
        op-2.json
```

Addons allow complex extensions to be attached without modifying the base object structure.

11.3 Addon Referencing

Addons are associated with the object directory in which they reside.

For example:

```
setups/set-1/addons/setup-addon-1.json
```

indicates an addon attached to setup `set-1`.

Similarly:

```
setups/set-1/operations/addons/op-2.json
```

represents an addon attached to operation `op-2`.

11.4 Extension Safety

Extensions and addons must not alter the interpretation of the core MIFX objects.

Systems must remain able to process the MIFX package even when unknown extensions are present.

This ensures that MIFX remains forward-compatible and vendor-neutral.

12 MIFX Package Format

A MIFX file is distributed as a compressed package containing the machining data and associated resources.

The package uses the file extension:

```
.mifx
```

Internally, a MIFX file is a standard ZIP archive containing a structured directory tree.

12.1 Root Structure

The root of the package contains the main job descriptor and the primary directories used by the MIFX format.

Example structure:

```
job.mifx
|
+-- job.json
|
+-- setups/
+-- tools/
+-- geometry/
+-- in_process/
\-- addons/
```

The only mandatory file at the root of the package is:

```
job.json
```

This file acts as the entry point for interpreting the MIFX package.

12.2 Directory Overview

The standard directories of a MIFX package are summarized below.

- **setups/**

  Contains machining setups and their associated operations.

- **tools/**

  Contains tool groups and tool definitions used by the machining process.

- **geometry/**

  Contains part, stock, fixture, and setup assembly geometry resources.

- **in_process/**

  Contains optional intermediate workpiece states generated by simulation or analysis.

- **addons/**

  Contains optional extensions provided by external systems.

Implementations may omit directories that are not required by the machining job.

12.3 Directory Flexibility

The MIFX specification defines the logical structure of the package but does not require every directory to be present.

Systems generating MIFX packages may include only the resources necessary for the intended use.

Consumers of MIFX packages must ignore directories or files they do not recognize.

12.4 Transport and Distribution

Because the MIFX package is a ZIP archive, it can be easily transmitted, stored, or versioned using existing tools.

Examples include:

- email attachments
- version control repositories
- manufacturing data exchanges
- digital job packages

This packaging approach ensures that all data required for interpreting the machining job can be distributed as a single file.

13 Compatibility and Forward Evolution

The MIFX specification is designed to support long-term compatibility across different systems and versions.

Implementations should follow the compatibility principles described in this section.

13.1 Unknown Fields

MIFX consumers must ignore JSON fields they do not recognize.

This allows new properties to be introduced without breaking existing implementations.

13.2 Unknown Objects

Systems encountering unknown objects or directories should ignore them unless they are required for core processing.

This enables future extensions of the format.

13.3 Extensions and Addons

Extensions and addons allow systems to attach additional information to MIFX objects without modifying the core structure.

Unknown extensions must not prevent a system from processing the package.

13.4 Version Identification

Each MIFX package declares its specification version using the `mifxVersion` field located in `job.json`.

Example:

```
"mifxVersion": "0.1"
```

Consumers may use this value to determine compatibility with the package structure.

13.5 Backward Compatibility

Future revisions of the MIFX specification should preserve the ability to interpret earlier versions whenever possible.

Changes that would invalidate existing packages should be avoided.

This principle ensures the long-term stability of the MIFX ecosystem.

A Minimal MIFX Package

This appendix shows the smallest possible valid MIFX package.

A.1 Directory Structure

```
job.mifx
|
+-- job.json
|
+-- setups/
|   \-- set-1/
|       +-- setup.json
|       \-- operations/
|           +-- op-1.json
|           \-- cldata/
|               \-- op-1.apt
|
\-- tools/
    +-- main.json
    \-- main/
        \-- 1.json
```

A.2 job.json

```
{
  "mifxVersion": "0.1",
  "header": {
    "job": {
      "name": "Minimal Example",
      "units": {
        "system": "metric",
        "linear": "mm"
      }
    }
  },
  "setups": [
    {
      "id": "set-1",
      "path": "setups/set-1/setup.json"
    }
  ],
  "tools": [
    {
      "role": "tool_group",
      "kind": "json",
```

```
      "path": "tools/main.json",
      "present": true
    }
  ]
}
```


A.3 setup.json


```
{
  "id": "set-1",
  "index": 1,
  "name": "Example Setup",
  "operations": [
    {
      "id": "op-1",
      "role": "toolpath",
      "kind": "mill",
      "path": "operations/op-1.json"
    }
  ]
}
```


A.4 op-1.json


```
{
  "id": "op-1",
  "name": "Face",
  "strategy": "face",
  "artifacts": [
    {
      "role": "tool",
      "kind": "json",
      "path": "../../../tools/main/1.json",
      "present": true
    },
    {
      "role": "toolpath",
      "kind": "apt",
      "path": "cldata/op-1.apt",
      "present": true
    }
  ]
}
```

B Milling Operation Example

The following example shows a milling operation using a face milling strategy.


```
{
  "id": "op-2",
  "name": "Face1",
  "strategy": "face",

  "artifacts": [
    {
      "role": "tool",
      "kind": "json",
      "path": "../../../tools/main/2.json",
      "present": true
    },
    {
      "role": "toolpath",
      "kind": "apt",
      "path": "cldata/op-2.apt",
      "present": true
    },
    {
      "role": "cutting_condition",
      "kind": "json",
      "path": "cutting_conditions/op-2.json",
      "present": true
    },
    {
      "role": "compensation",
      "kind": "json",
      "path": "compensation/op-2.json",
      "present": true
    }
  ]
}
```

C Tool Definition Example

The following example defines a bull mill mounted in a 45 degree angle head.

```
{
  "id": "7",
  "unit": "in",
  "description": "1/2 Bull Mill mounted in 45 degree angle head",

  "artifacts": [
    {
      "role": "tool_profile",
      "kind": "json",
      "path": "profiles/7.json",
      "present": true
    },
    {
      "role": "tool_mount",
      "kind": "json",
      "path": "mounts/7.json",
      "present": true
    },
    {
      "role": "tool_controls",
      "kind": "json",
      "path": "controls/7.json",
      "present": true
    },
    {
      "role": "tool_extensions",
      "kind": "json",
      "path": "extensions/7.json",
      "present": true
    }
  ]
}
```

D Complete MIFX Package Structure

The following example illustrates the complete directory structure of a MIFX package.

```
job.mifx
|
+-- job.json
|
+-- setups/
|   +-- set-1/
|   |   +-- setup.json
|   |   +-- notes.json
|   |   +-- offset.json
|   |   +-- part.json
|   |   +-- stock.json
|   |   +-- fixture.json
|   |   +-- setup-asm.json
|   |   |
|   |   +-- addons/
|   |   |   \-- setup-addon-1.json
|   |   |
|   |   \-- operations/
|   |       +-- op-1.json
|   |       +-- op-2.json
|   |       |
|   |       +-- cldata/
|   |       |   +-- op-1.apt
|   |       |   \-- op-2.apt
|   |       |
|   |       +-- notes/
|   |       +-- csys/
|   |       +-- offsets/
|   |       +-- compensation/
|   |       +-- cutting_conditions/
|   |       +-- extensions/
|   |       |
|   |       \-- addons/
|   |           +-- op-1.json
|   |           \-- op-2.json
|   |
|   \-- set-2/
|       +-- setup.json
|       \-- operations/
|
+-- tools/
```

```
|   +-- main.json
|   |
|   \-- main/
|       +-- 1.json
|       +-- 2.json
|       |
|       +-- profiles/
|       |   +-- 1.json
|       |   \-- 2.json
|       |
|       +-- mounts/
|       +-- controls/
|       |   +-- 1.json
|       |   \-- 2.json
|       |
|       +-- meshes/
|       +-- extensions/
|       |   +-- 1.json
|       |   \-- 2.json
|       |
|       \-- addons/
|           +-- 1.json
|           \-- 2.json
|
+-- geometry/
|   +-- setups/
|   +-- parts/
|   +-- stock/
|   \-- fixtures/
|
+-- in_process/
|   +-- op1-5.json
|   +-- op6-10.json
|   +-- meshes/
|   +-- extensions/
|   \-- addons/
|
\-- addons/
    \-- index.json
```